ML-PROJECT

Prediction of Approval of Consumer Personal Loans Applications using different Machine Learning Algorithms.

FINAL PROJECT REPORT ALONG WITH CODE

DONE BY:

RAHUL K

DATA CLEANING:

1. Understanding the information regarding the given data

```
#Importing the library
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from scipy import stats
import pandas as pd

#Importing the dataset
df = pd.read_csv('Proj 4-Train Data.csv')
df_1=pd.read_csv('Proj 4-CleanTestData.csv')


df.info()
```

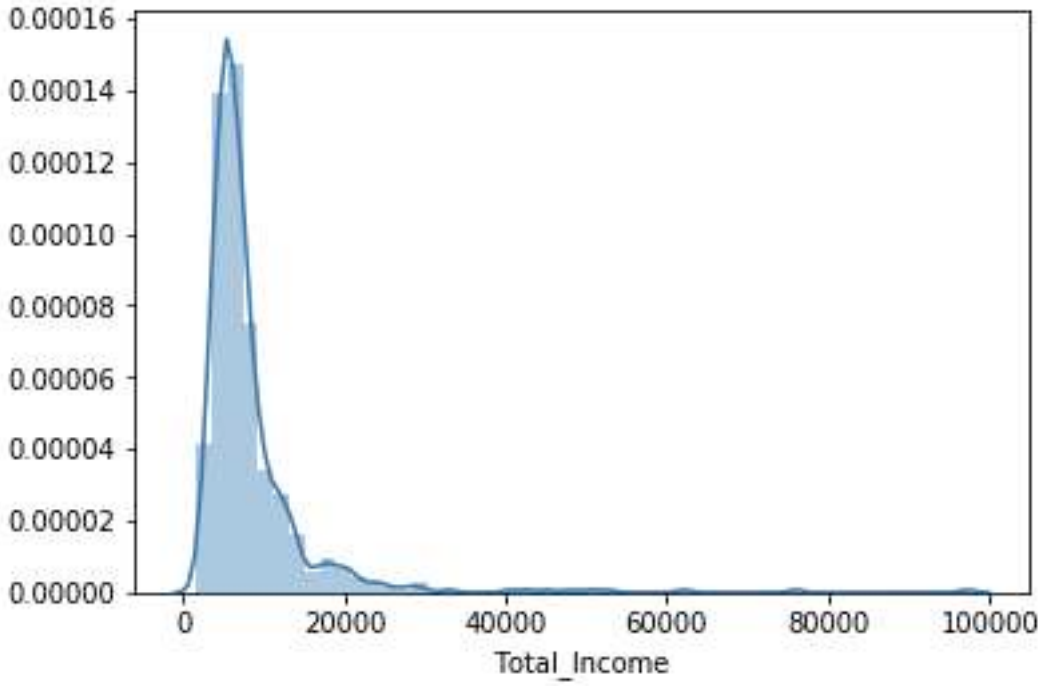
```
 #   Column           Non-Null Count  Dtype
---  ------           --------------  -----
 0   Loanapp_ID       614 non-null    object
 1   Sex              601 non-null    object
 2   Marital_Status   611 non-null    object
 3   first_name       614 non-null    object
 4   last_name        614 non-null    object
 5   email            614 non-null    object
 6   address          614 non-null    object
 7   Dependents       599 non-null    object
 8   Qual_var         614 non-null    object
 9   SE               582 non-null    object
 10  App_Income_1     614 non-null    float64
 11  App_Income_2     614 non-null    float64
 12  CPL_Amount       612 non-null    float64
 13  CPL_Term         600 non-null    float64
 14  Credit_His       564 non-null    float64
 15  Prop_Area        614 non-null    object
 16  INT_ID           614 non-null    float64
 17  Prev_ID          614 non-null    object
 18  AGT_ID           614 non-null    object
```

```
df.describe()
```

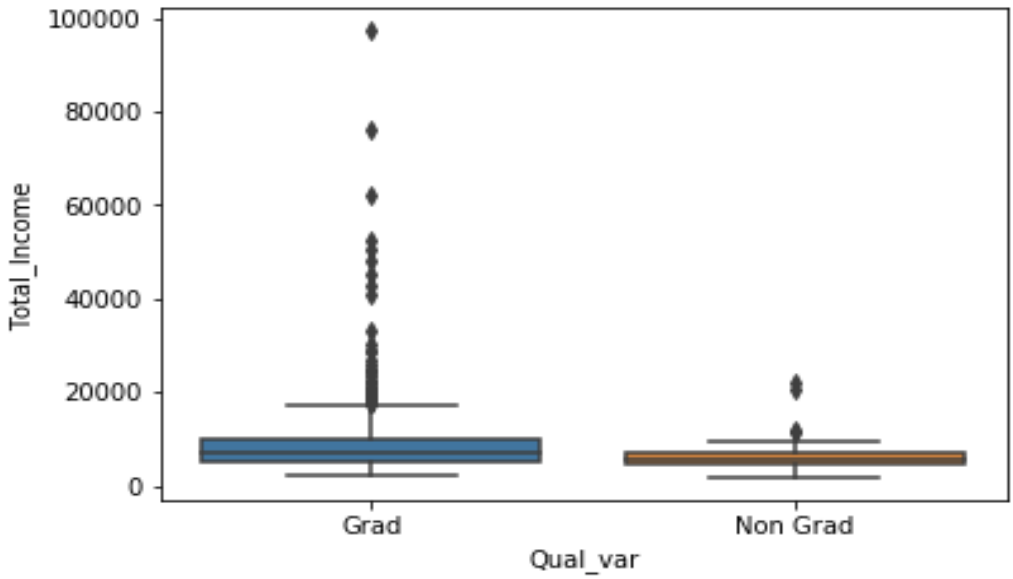
```
In [3]: df.describe()
Out[3]:
       App_Income_1  App_Income_2  ...  Unnamed: 33  Unnamed: 34
count    614.000000    614.000000  ...          0.0          0.0
mean    6484.151140   1945.494958  ...          NaN          NaN
std     7330.850008   3511.498043  ...          NaN          NaN
min      180.000000      0.000000  ...          NaN          NaN
25%     3453.000000      0.000000  ...          NaN          NaN
50%     4575.000000   1426.200000  ...          NaN          NaN
75%     6954.000000   2756.700000  ...          NaN          NaN
max    97200.000000  50000.400000  ...          NaN          NaN

[8 rows x 21 columns]
```

```
#Data Preprocessing
#Removing unwanted columns
df.drop(df.filter(regex="Unname"),axis=1,
inplace=True)

df=df.dropna(axis=0, subset=['Loanapp_ID'])

df.drop(['first_name','last_name','email','addres
s','AGT_ID','INT_ID','Prev_ID'],axis=1,inplace=Tr
ue)

#Resetting index to Loanapp_ID
df.set_index('Loanapp_ID', inplace=True)

#Handling numerical missing data
#Here we fill the numerical missing values by the
median and categorical by most frequent

from sklearn .impute import SimpleImputer
imputer1 =
SimpleImputer(missing_values=np.nan,strategy="med
ian")
imputer1.fit(df.iloc[:,7:9])
df.iloc[:,7:9]=imputer1.transform(df.iloc[:,7:9])


imputer2 =
SimpleImputer(missing_values=np.nan,strategy="mos
```

```
t_frequent")
imputer2.fit(df.iloc[:,0:5])
df.iloc[:,0:5]=imputer2.transform(df.iloc[:,0:5])

df['Credit_His'].value_counts()
imputer2.fit(df.iloc[:,9:10])
df.iloc[:,9:10]=imputer2.transform(df.iloc[:,9:10
])

#Merging the two incomes into a single one
df["App_Income_1"]=df["App_Income_1"]+df["App_Inc
ome_2"]
df.drop("App_Income_2",axis=1,inplace=True)
df.rename(columns=
{'App_Income_1':'Total_Income'},inplace=True)


#Checking Distribution of Numeric values

sns.distplot(df["Total_Income"])
```

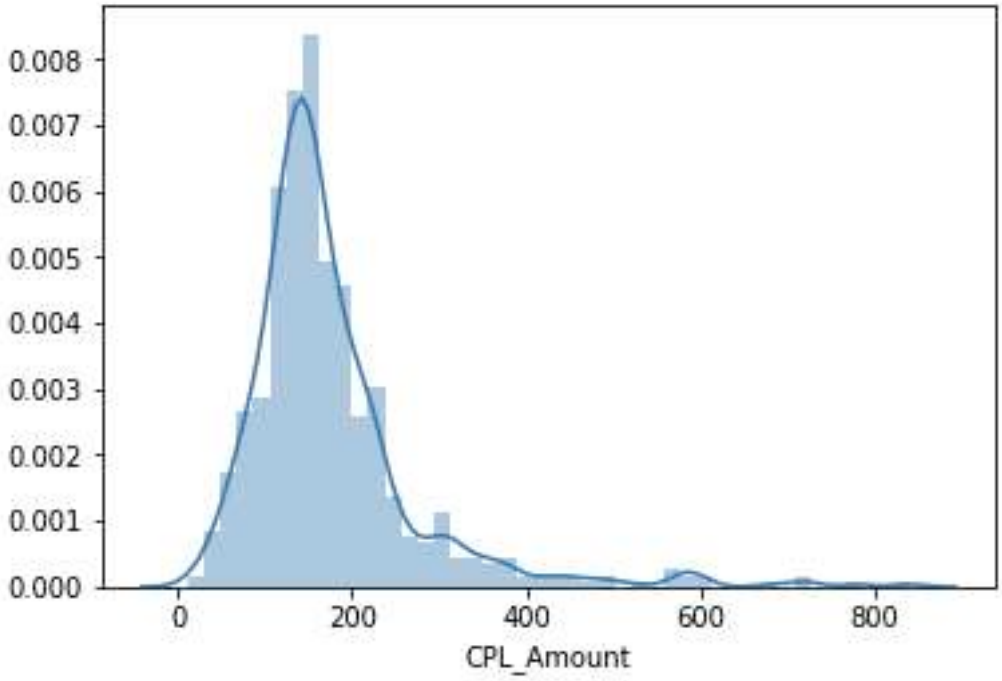



```
sns.boxplot(y="Total_Income",x=
"Qual_var",data=df)
```



The income of Grad applicants varies a lot as compared to the Non Grad applicants. So we use median to fill the missing values for better accuracy.

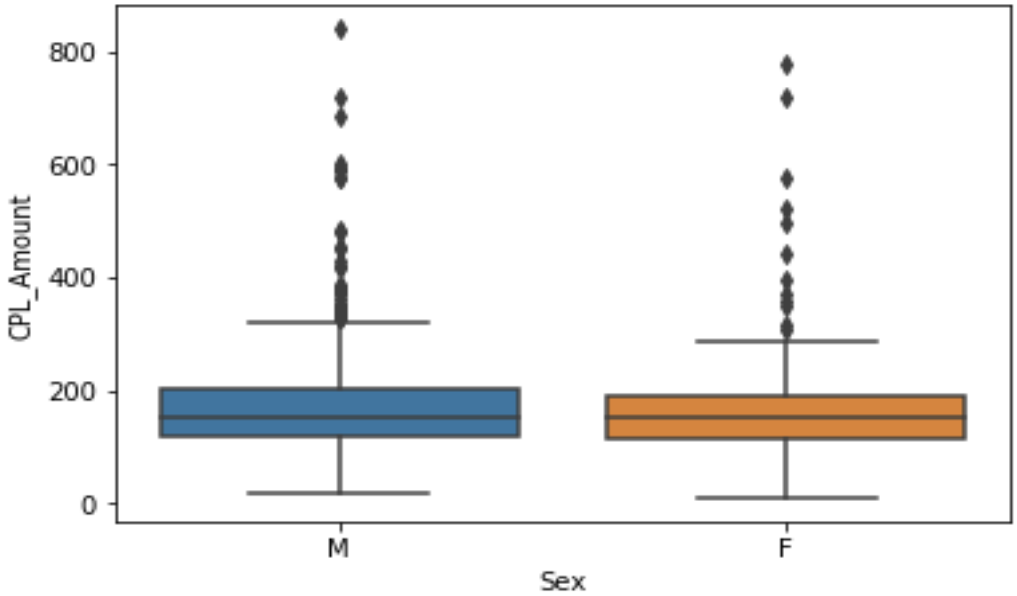
```
sns.distplot(df["CPL_Amount"])
```



Here we can observe that the CPL_amount for majority of applicants is around 190.

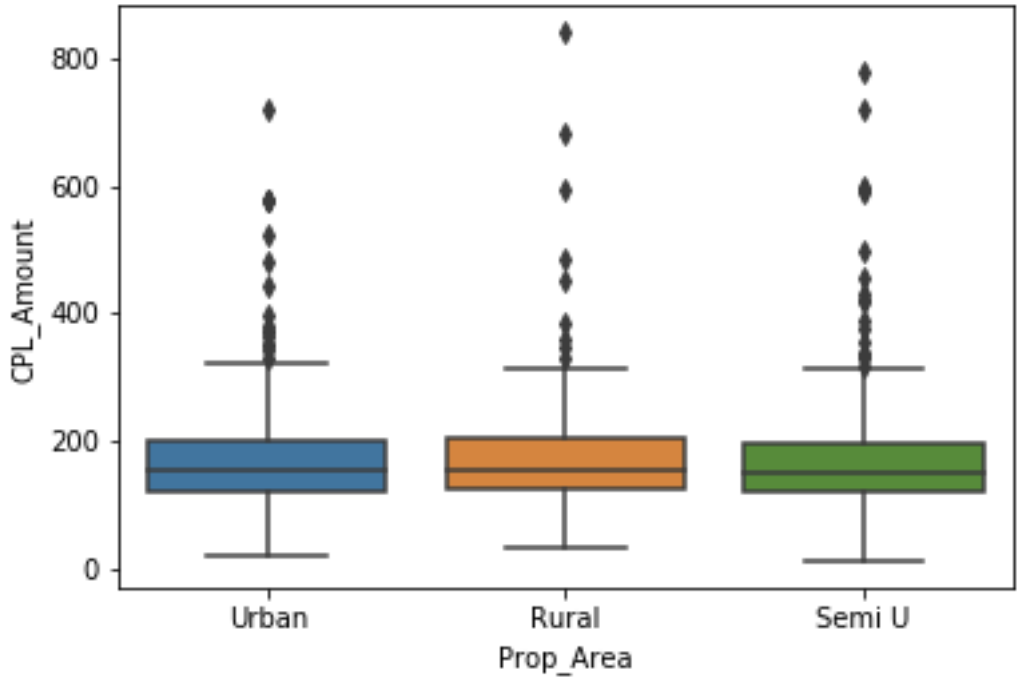
```
sns.boxplot(y="CPL_Amount",data=df)
```

```
sns.boxplot(y="CPL_Amount",x= "Sex",data=df)
```



Here we can observe that the characteristics of CPL_Amount doesn't depend on the Sex as both have the same range and distribution.

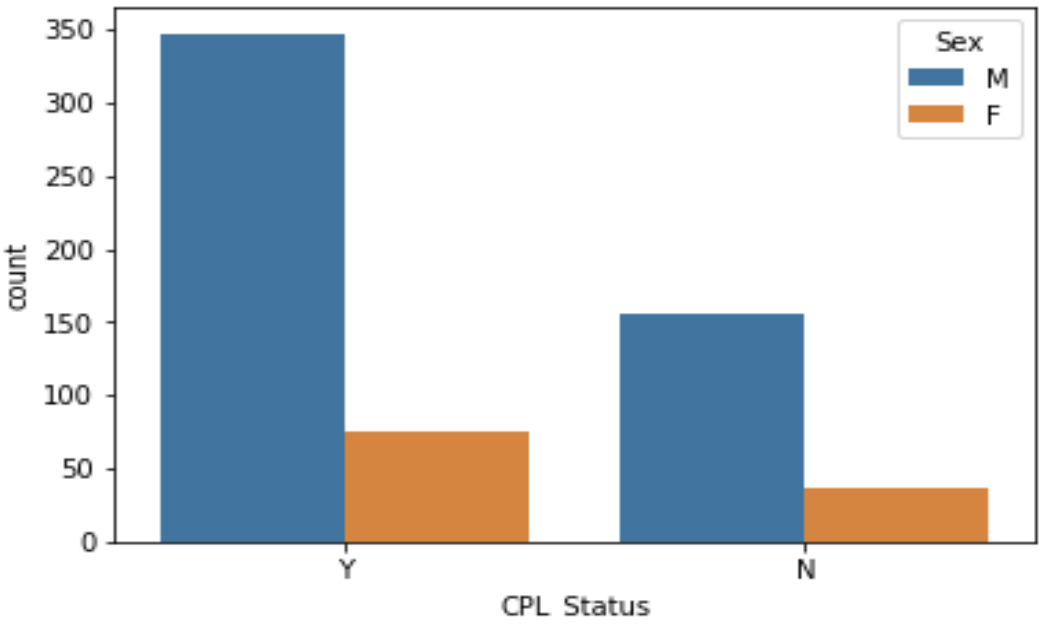
```
sns.boxplot(y="CPL_Amount",x="Prop_Area",data=df)
```



Similarly we can also observe that the characteristics of CPL_Amount doesn't depend on the prop_Area as all of the applicants apply for more or less the same range of CPL_Amount.

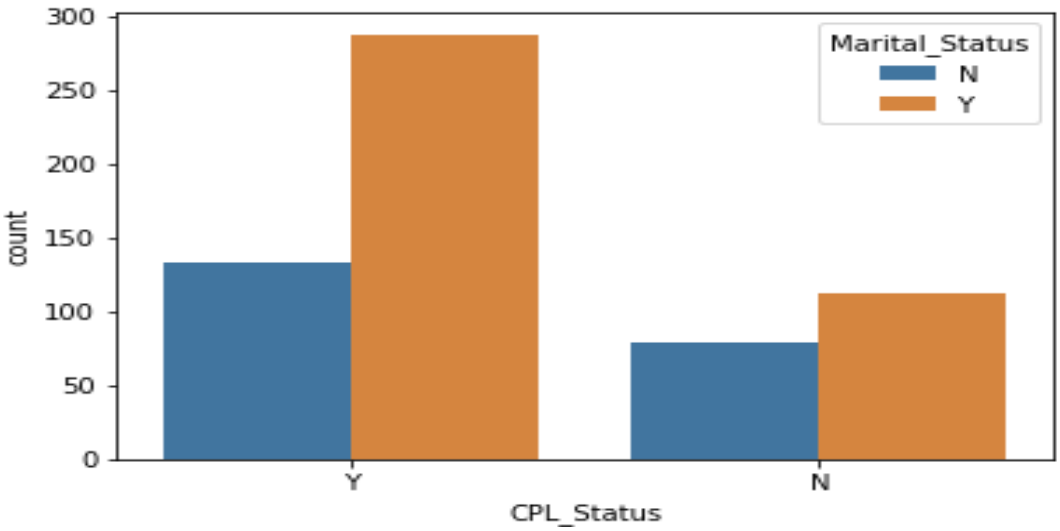
PERFORMING EXPLOITARY DATA ANALYSIS:

```
#Pure insights from the data and understanding it
1.
sns.countplot(x="CPL_Status",data=df,hue="Sex")
```



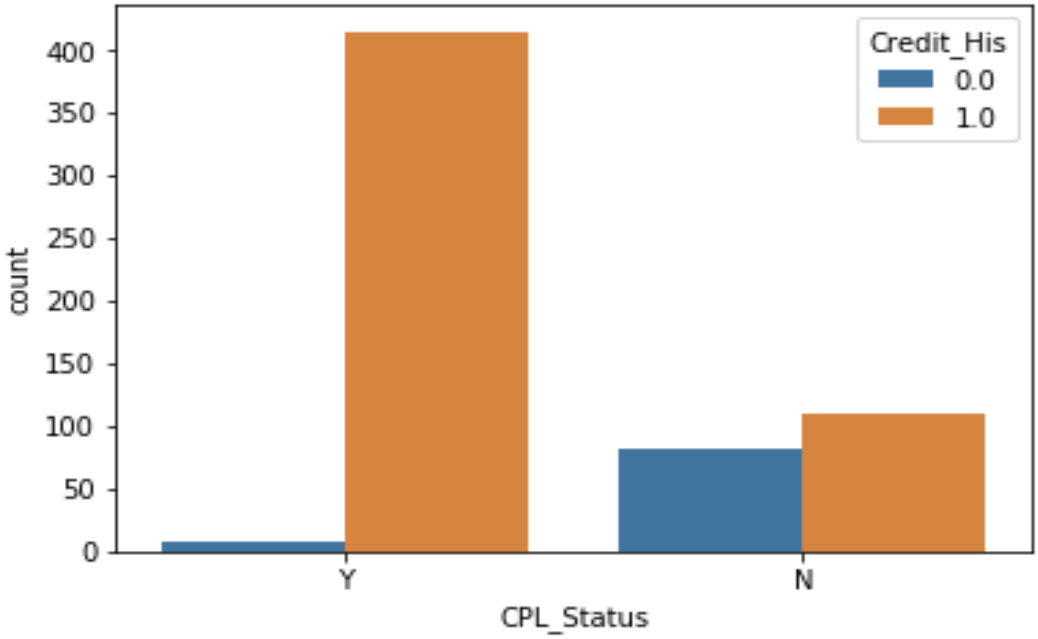
From this plot we can observe that the no. of male applicants are more than the female applicants but it doesn't have any impact in the acceptance rate though(CPL_Status).

```
2.
sns.countplot(x="CPL_Status",data=df,hue="Marita
_Status")
```



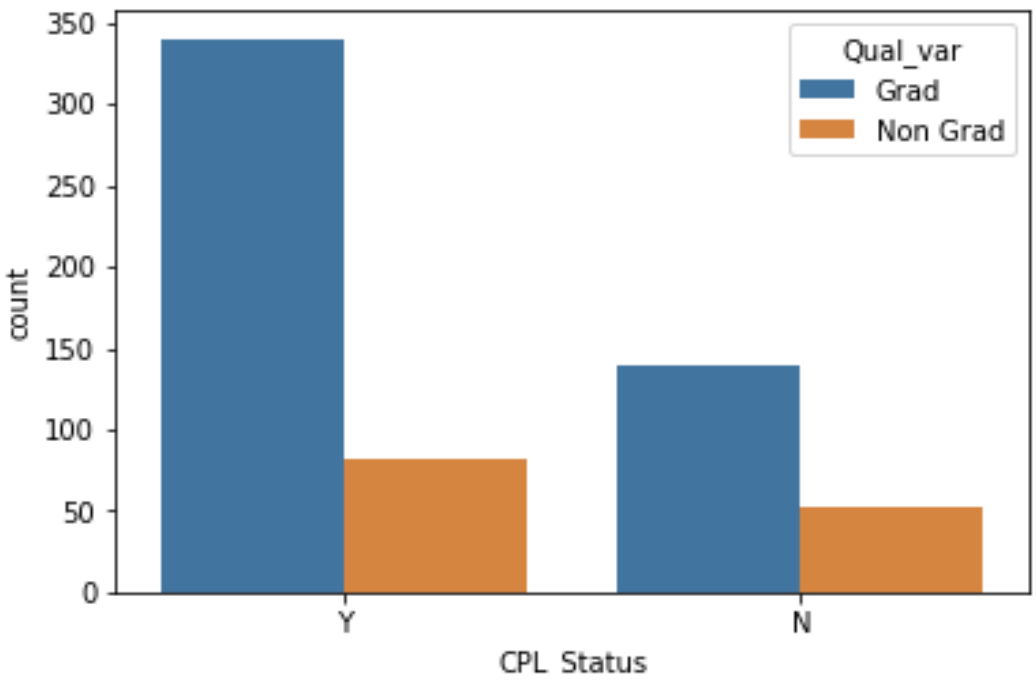
From this above plot we can observe that the most of the applicants who have applied for loans are married and their acceptance rate is also a bit high might be because of the co-applicants income.

3.
```
sns.countplot(x="CPL_Status",data=df,hue="Credit_His")
```



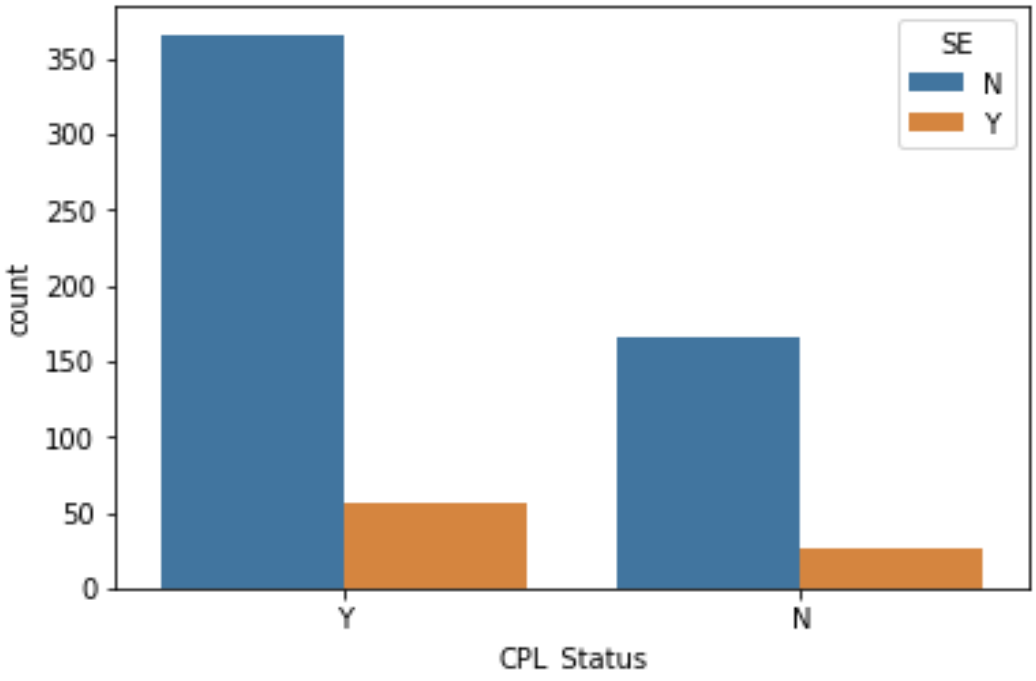
Here we observe a very important feature, which is the credit history. As we can see from the above plot the banks prefer or approve the loans of applicants who have a credit history. So as a result the applicants who have a credit history have more probability of getting loan approved compared to those who do not have any kinds of credit history

4.
```
sns.countplot(x="CPL_Status",data=df,hue="Qual_var")
```



Here we observe that first of all the applicants who have graduated are more than those who are non-graduated. And then there is a slightly more probability of getting the loan approved if you are a graduate comparatively.

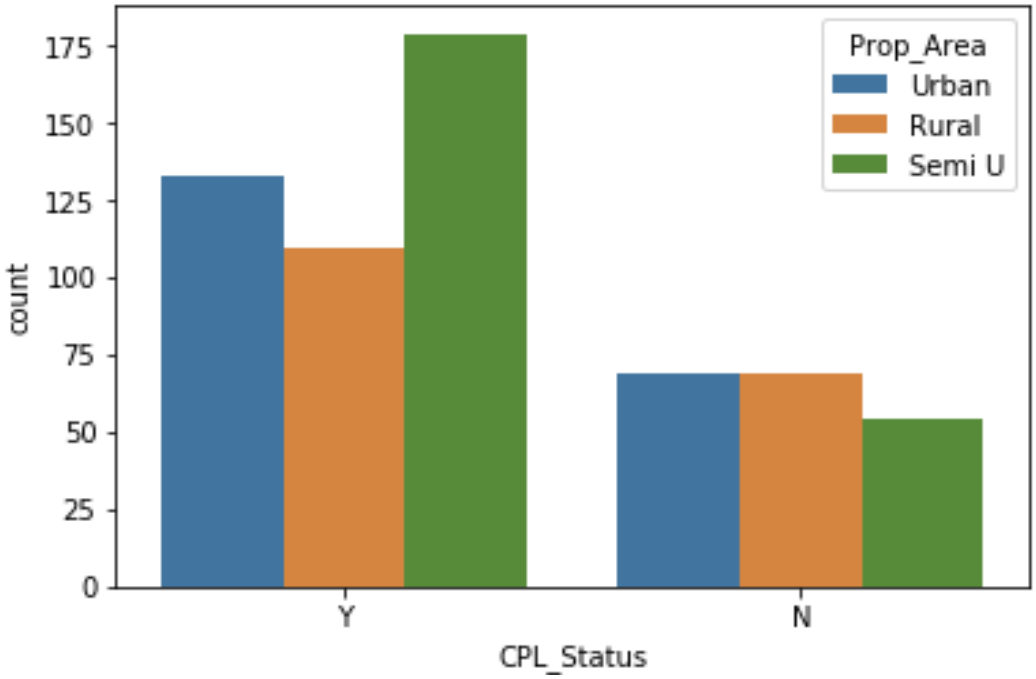
5.
```
sns.countplot(x="CPL_Status",data=df,hue="SE")
```



We can observe that most of the applicants are salaried people but that doesn’t have any effect on the CPL_Status and they is a equal chance of approval whether you are SE or not.

6.

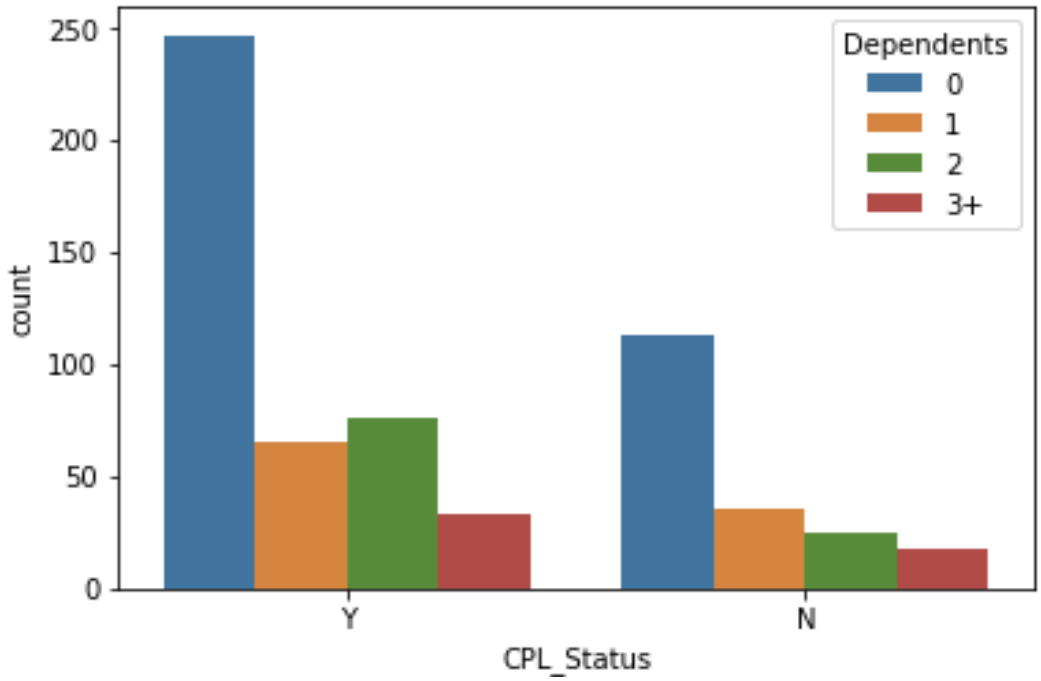
```
sns.countplot(x="CPL_Status",data=df,hue="Prop_Area")
```



What we can observe from here is that the loan approval probability is slightly higher for the people in Urban and Semi Urban compared to those in Rural might be because of higher salary or due to the grad status.

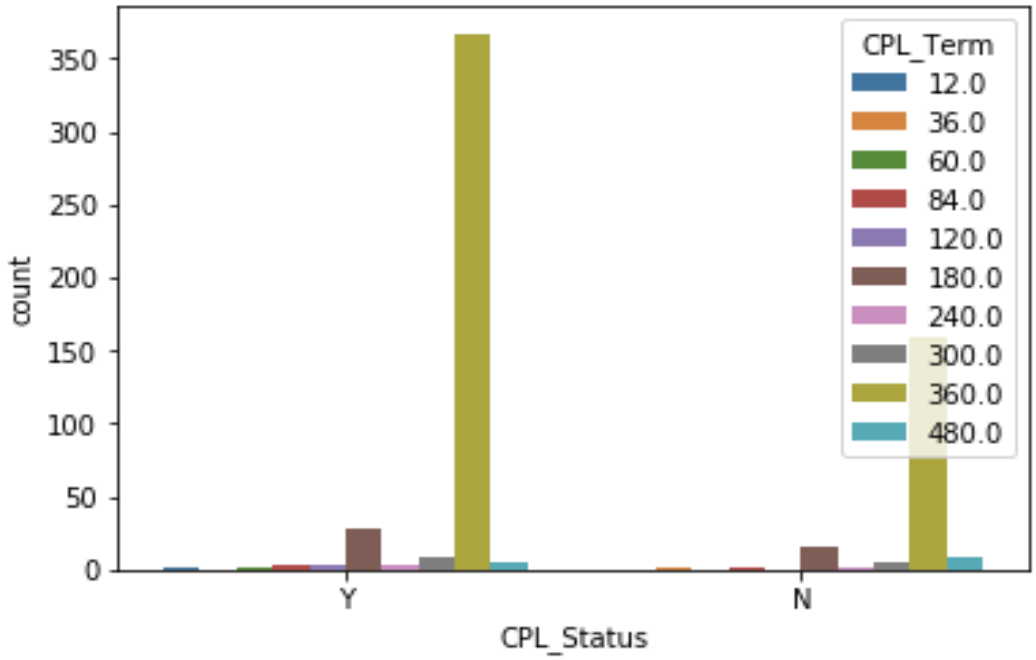
7.

```
sns.countplot(x="CPL_Status",data=df,hue="Dependents")
```



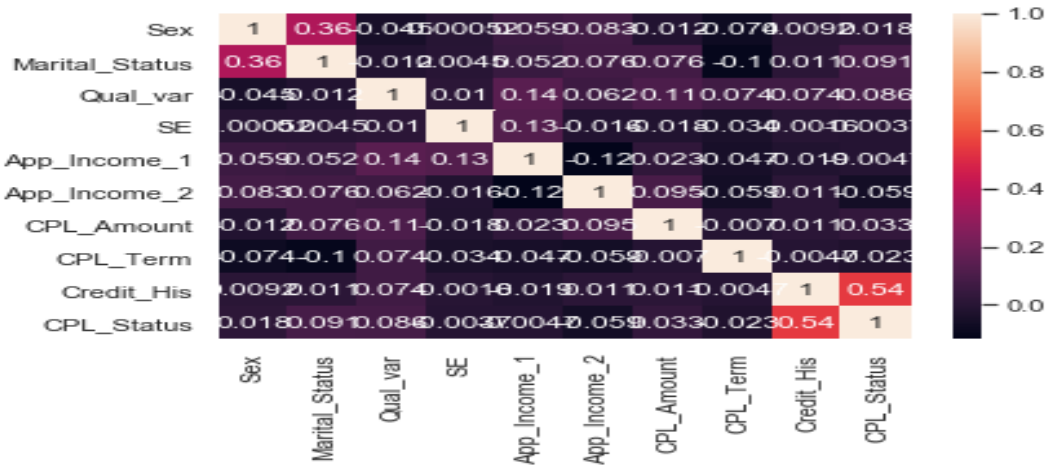
Clearly the applicants with 0 dependents are much higher compared to other applicants. The more no of dependents the less likely to get the loan approved but it can compensated by factors like higher salary, grad and good credit history.

8.
```
sns.countplot(x="CPL_Status",data=df,hue="CPL_Term")
```



Clearly Most of the applicants have chosen the term 360 for the repayment of the loans followed by 180.And this parameter doesn’t really have any significance individually.

```
#Label encoding no of dependents and prop Area
from sklearn.preprocessing import LabelEncoder
le = LabelEncoder()

df['SE']= le.fit_transform(df['SE']) #Y:1 N:0
df["Marital_Status"]=
le.fit_transform(df["Marital_Status"]) #Y:1   N:0
df['Sex']= le.fit_transform(df['Sex']) #M:1 F:0
df['Dependents']=
le.fit_transform(df['Dependents'])
df['Prop_Area']=
le.fit_transform(df['Prop_Area']) #Urban:2
Rural:0 Semi-Urban:1
df["Qual_var"]=le.fit_transform(df["Qual_var"])
#Grad:0  NonGrad:1
df['CPL_Status']=
le.fit_transform(df['CPL_Status'])#yes:1 No:0

#Data visualization
correlation=df.corr()
sns.heatmap(correlation,xticklabels=correlation.c
olumns,yticklabels=correlation.columns,annot=True
)
sns.pairplot(df)
```



From the heat map we can observe that the correlation coefficient of (CPL_Status,SE)=0.003 which is very less compared to other correlation values. So this feature does not play any important role in predicting the CPL_Status hence we drop it.

```
df.drop("SE",axis=1,inplace=True)

#Avoiding the famous dummy variable trap ie One
hot encoding the categorical variables Dependents
and Prop_Area

df=pd.get_dummies(df,columns=["Dependents","Prop_
Area"],drop_first=True)


#Splitting into train and validation here.
#Note here test refers to validation and not the
actual test data.

from sklearn.model_selection import
train_test_split
X = df.loc[:, df.columns != 'CPL_Status']
y = df.iloc[:,7].values
X=X.to_numpy()
y=y.to_numpy()
X_train, X_test, y_train, y_test =
train_test_split(X, y,
random_state=0,test_size=0.25)


#Applying feature scaling to
CPL_Amount,CPL_Term,Total_Income
from sklearn.preprocessing import StandardScaler
sc_X = StandardScaler()

X_train[:,[3,4,5]] =
sc_X.fit_transform(X_train[:,[3,4,5]])
X_test[:,[3,4,5]] =
sc_X.transform(X_test[:,[3,4,5]])
```

TRAINING DIFFERENT MODELS AND COMPARING THE ACCURACIES.

1. Decision Trees

```
# Fitting Decision Tree Classification to the
Training set
from sklearn.tree import DecisionTreeClassifier
classifierDT = DecisionTreeClassifier(criterion =
'entropy',
max_depth=40,min_samples_split=40,random_state =
0)

classifierDT.fit(X_train, y_train)

# Predicting the Test(validation) set results
y_predDT = classifierDT.predict(X_test)

# Making the Confusion Matrix
from sklearn.metrics import confusion_matrix
from sklearn.metrics import accuracy_score
from sklearn.metrics import classification_report


cm = confusion_matrix(y_test, y_predDT)
print("Confusion matrix")
print(cm)
print('Accuracy Score
:',accuracy_score(y_test,y_predDT))
print('Report :')
print(classification_report(y_test, y_predDT))

#Prediction of the actual testset:
y_actDT = classifierDT.predict(X_act)

df_1["DT_OUT"]=y_actDT
```

```
Confusion matrix
[[ 21  22]
 [  9 102]]
Accuracy Score : 0.7987012987012987
Report :
              precision    recall  f1-score   support

           0       0.70      0.49      0.58        43
           1       0.82      0.92      0.87       111

    accuracy                           0.80       154
   macro avg       0.76      0.70      0.72       154
weighted avg       0.79      0.80      0.79       154
```

2. Random Forest

```
# Fitting Random Forest Classification to the
Training set
from sklearn.ensemble import
RandomForestClassifier
classifierRF =
RandomForestClassifier(n_estimators =
100,max_depth=30,min_samples_split=20, criterion
= 'entropy', random_state = 0)
classifierRF.fit(X_train, y_train)
# Predicting the Test set results
y_predRF = classifierRF.predict(X_test)

# Making the Confusion Matrix
from sklearn.metrics import confusion_matrix
from sklearn.metrics import accuracy_score
from sklearn.metrics import classification_report

cm = confusion_matrix(y_test, y_predRF)
print("Confusion matrix")
print(cm)
print('Accuracy Score
:',accuracy_score(y_test,y_predRF))
print('Report :')
print(classification_report(y_test, y_predRF))

#Prediction of the actual testset:
```

```
y_actRF = classifierRF.predict(X_act)
df_1["RF_OUT"]=y_actRF
```

```
Confusion matrix
[[ 19  24]
 [  2 109]]
Accuracy Score : 0.8311688311688312
Report :
              precision    recall  f1-score   support

           0       0.90      0.44      0.59        43
           1       0.82      0.98      0.89       111

    accuracy                           0.83       154
   macro avg       0.86      0.71      0.74       154
weighted avg       0.84      0.83      0.81       154
```

3. SVM

```
# Fitting Kernel SVM to the Training set
from sklearn.svm import SVC
classifierSVM = SVC(kernel = 'rbf',random_state =
0)
classifierSVM.fit(X_train, y_train)

# Predicting the Test set results
y_predSVM = classifierSVM.predict(X_test)

# Making the Confusion Matrix
from sklearn.metrics import confusion_matrix
from sklearn.metrics import accuracy_score
from sklearn.metrics import classification_report

cm = confusion_matrix(y_test, y_predSVM)
print("Confusion matrix")
print(cm)
print('Accuracy Score
:',accuracy_score(y_test,y_predSVM))
print('Report :')
print(classification_report(y_test, y_predSVM))

#Prediction of the actual testset:
```

```
y_actSVM = classifierSVM.predict(X_act)
df_1["RF_OUT"]=y_actSVM
```

```
Confusion matrix
[[ 19  24]
 [  1 110]]
Accuracy Score : 0.8376623376623377
Report :
              precision    recall  f1-score   support

           0       0.95      0.44      0.60        43
           1       0.82      0.99      0.90       111

    accuracy                           0.84       154
   macro avg       0.89      0.72      0.75       154
weighted avg       0.86      0.84      0.82       154
```

4. KNN

```
#Using KNN:
from sklearn.neighbors import
KNeighborsClassifier
classifierKNN = KNeighborsClassifier(n_neighbors
= 5, metric = 'minkowski',algorithm="auto", p =
2)
classifierKNN.fit(X_train, y_train)

y_predKNN = classifierKNN.predict(X_test)
# Making the Confusion Matrix
from sklearn.metrics import confusion_matrix
from sklearn.metrics import accuracy_score
from sklearn.metrics import classification_report

cm = confusion_matrix(y_test, y_predKNN)
print("Confusion matrix")
print(cm)
print('Accuracy Score
:',accuracy_score(y_test,y_predKNN))
print('Report :')
print(classification_report(y_test, y_predKNN))
```

```
#Prediction of the actual testset:
y_actKNN = classifierKNN.predict(X_act)
df_1["KNN_OUT"]=y_actKNN
```

```
Confusion matrix
[[ 19  24]
 [  5 106]]
Accuracy Score : 0.8116883116883117
Report :
              precision    recall  f1-score   support

           0       0.79      0.44      0.57        43
           1       0.82      0.95      0.88       111

    accuracy                           0.81       154
   macro avg       0.80      0.70      0.72       154
weighted avg       0.81      0.81      0.79       154
```

5. Logistic Regression

```
# Fitting Logistic Regression to the Training set
from sklearn.linear_model import
LogisticRegression
classifierLR = LogisticRegression(random_state =
0)
classifierLR.fit(X_train, y_train)

# Predicting the Test set results
y_predLR = classifierLR.predict(X_test)


# Making the Confusion Matrix
from sklearn.metrics import confusion_matrix
from sklearn.metrics import accuracy_score
from sklearn.metrics import classification_report

cm = confusion_matrix(y_test, y_predLR)
print("Confusion matrix")
print(cm)
print('Accuracy Score
:',accuracy_score(y_test,y_predLR))
print('Report :')
```

```
print(classification_report(y_test, y_predLR))

#Prediction of the actual testset:
y_actLR = classifierLR.predict(X_act)
df_1["LR_OUT"]=y_actLR
```

```
Confusion matrix
[[ 19  24]
 [  2 109]]
Accuracy Score : 0.8311688311688312
Report :
              precision    recall  f1-score   support

           0       0.90      0.44      0.59        43
           1       0.82      0.98      0.89       111

    accuracy                           0.83       154
   macro avg       0.86      0.71      0.74       154
weighted avg       0.84      0.83      0.81       154
```

6. Multi Layer Perceptron

```
# Importing the MLPClassifier libraries and
packages
from sklearn.neural_network import MLPClassifier
mlp = MLPClassifier(hidden_layer_sizes=(6,6),
activation='relu', solver='adam',
max_iter=500,random_state=1)
mlp.fit(X_train,y_train)

# Predicting the Test set results
y_predMLP = mlp.predict(X_test)
y_predMLP = (y_pred > 0.5)

# Making the Confusion Matrix
from sklearn.metrics import confusion_matrix
from sklearn.metrics import accuracy_score
from sklearn.metrics import classification_report

cm = confusion_matrix(y_test, y_predMLP)
print("Confusion matrix")
print(cm)
```

```
print('Accuracy Score
:',accuracy_score(y_test,y_predMLP))
print('Report :')
print(classification_report(y_test, y_predMLP))
#Prediction of the actual testset:
y_actMLP = mlp.predict(X_act)
df_1["MLP_OUT"]=y_actMLP
```

```
Confusion matrix
[[ 21  22]
 [  2 109]]
Accuracy Score : 0.8441558441558441
Report :
              precision    recall  f1-score   support

           0       0.91      0.49      0.64        43
           1       0.83      0.98      0.90       111

    accuracy                           0.84       154
   macro avg       0.87      0.74      0.77       154
weighted avg       0.85      0.84      0.83       154
```

COMPARING THE ACCURACIES OF DIFFERENT ALGORITHMS:

S.No	Algorithm	Accuracy
1.	Decision Tree	**79.9%**
2.	Random Forest	**83.1%**
3.	SVM	**83.8%**
4.	KNN	**81.2%**
5.	Logistic Regression	**83.1%**
6.	MLP	**84.4%**

Note: These accuracies can be further improved by using hyperparameter tuning in each of the model.

The final output by different algorithms is appended in df_1 itself.